Documentation Reference

Follow the official Fly4Future documentation up to the section "Run your own simulation"
Stop at "Run your own simulation"
You do not need to follow their example simulations (e.g., moving the drone in a square path). We'll write our own simulation code instead.
Fly4Future Simulation Docs

GitHub Repository for examples:
https://github.com/ctu-mrs/mrs_core_examples

Drone Features documentation: https://ctu-mrs.github.io/docs/introduction/

ROS Tutorials: https://wiki.ros.org/ROS/Tutorials

Rospy: https://wiki.ros.org/rospy

Nvidia GPU I have: NVIDIA GeForce RTX 3050

Git Commands

git add .
git commit -m "Your commit message"
git push origin *branch-name*

START HERE:

- **Location of the One drone / three drone file:**
  opt/ros/noetic/share/mrs_uav_gazebo_simulation/tmux/one_drone

  Launching the One-Drone Simulation

  Imp: First Clone GitHub Repository:
  https://github.com/ctu-mrs/mrs_core_examples
  This has the sweeping generator code of the one drone in python
  Clone this in your catkin_ws/src

  Make sure you do catkin build in src location of catkin_ws

  1. **Start the One-Drone Simulation**
  (location:opt/ros/noetic/share/mrs_uav_gazebo_simulation/tmux/one_drone)
  S roscd mrs_uav_gazebo_simulation
  S cd tmux/one_drone
  S ./start.sh

2. **Open a new terminal tab (Ctrl + T), and run the sweeping generator:**
roslaunch example_sweeping_generator sweeping_generator.launch

3. Open another terminal tab (Ctrl + T), and list ROS services:
S rosservice list

4. Start the sweeping pattern:

S rosservice call /uav1/sweeping_generator/start 3.0
The drone should now begin a sweeping flight pattern.

5. To kill all nodes
Press " Cntl + a "
Then " k "
Then press " 9 "

Launching the Three-Drone Simulation

Clone the below github example into your catkin_ws/src
https://github.com/RishitaChawla/Drone_Research.git

After cloning, make sure you do catkin build or catkin_make in catkin_ws location.
Then
S cd catkin_ws/devel
S source setup.bash { done when there is change in script of code }

(location:opt/ros/noetic/share/mrs_uav_gazebo_simulation/tmux/one_drone)
1. **Start the three-drone simulation:**

S roscd mrs_uav_gazebo_simulation
S cd tmux/three_drones
S ./start.sh

2. Open a new terminal tab (Ctrl + T), navigate to the coordination launch file:
Location: catkin_ws/src/multi_uav_Coordination

S roscd example_multi_uav_coordination
S roslaunch example_multi_uav_coordination multi_uav_coordination.launch

3. Open another terminal tab (Ctrl + T), and start coordination for each drone:
The below three lines are one command

```
S rosservice call /uav1/multi_uav_coordination/start_in 1.0 & \
rosservice call /uav2/multi_uav_coordination/start_in 1.0 & \
rosservice call /uav3/multi_uav_coordination/start_in 1.0 & \
wait
```
Tip: Check available topics using: rostopic list | grep rgbd

You will see three drones taking their own trajectories.

4. Open another terminal
```
S rqt
```
Find the settings so that the camera view is visible

Custom World: charlotte.world
The charlotte.world file is a custom simulation environment tailored for disc detection experiments.

Key Features:
Strategically placed discs for UAV detection/tracking

Urban-like structures and buildings

Full compatibility with Fly4Future + MRS UAV framework

Installing charlotte.world

1. Download charlotte.world, and copy it into the correct directory:
https://drive.google.com/file/d/1Ckpf4sn3eGsjPgC_UPE0kG5H0xbGgjQJ/view?usp=drive_link

Corrected command:
```
sudo cp ~/catkin_ws/src/worlds/charlotte.world
/opt/ros/noetic/share/mrs_gazebo_common_resources/worlds/
```

2. Update session.yml to Use charlotte.world
Open the session file:

```
S roscd mrs_uav_gazebo_simulation
S cd tmux/three_drones
S sudo nano session.yml
```

3. Modify the session.yml file:
Find the line containing grass_plane (use Ctrl + F in nano)

Replace it with:
world_name: charlotte

3. Save and exit:
Press Ctrl + O, then Enter to save.

Press Ctrl + X to exit.

For Camera to work for disc detection:

1. Location: /opt/ros/noetic/share/mrs_uav_deployment/config/communication_config.yaml
   Copy and paste this in the communication_config file

```
topics: # topics published out of this machine to receivers specified above
 - name: "control_manager/mpc_tracker/diagnostics"
   compress: true # enable bz2 compression
   rate: 1.0
 - name: "control_manager/diagnostics"
   compress: true # enable bz2 compression
   rate: 1.0
 - name: "uav_manager/diagnostics"
   compress: true # enable bz2 compression
   rate: 1.0
 - name: "control_manager/mpc_tracker/predicted_trajectory"
   compress: true # enable bz2 compression
   rate: 2.0
 - name: "mrs_uav_status/uav_status"
   compress: true # enable bz2 compression
   rate: 1.0
 - name: "mrs_uav_status/uav_status_short"
   compress: true # enable bz2 compression
   rate: 10.0
 - name: "disc_detection"
   compress: false
   rate: 10.0  # 10 Hz is sufficient for detection events
   queue_size: 5

services:
  - name: "trajectory_generation/path"
    type: mrs_msgs/PathSrv
```

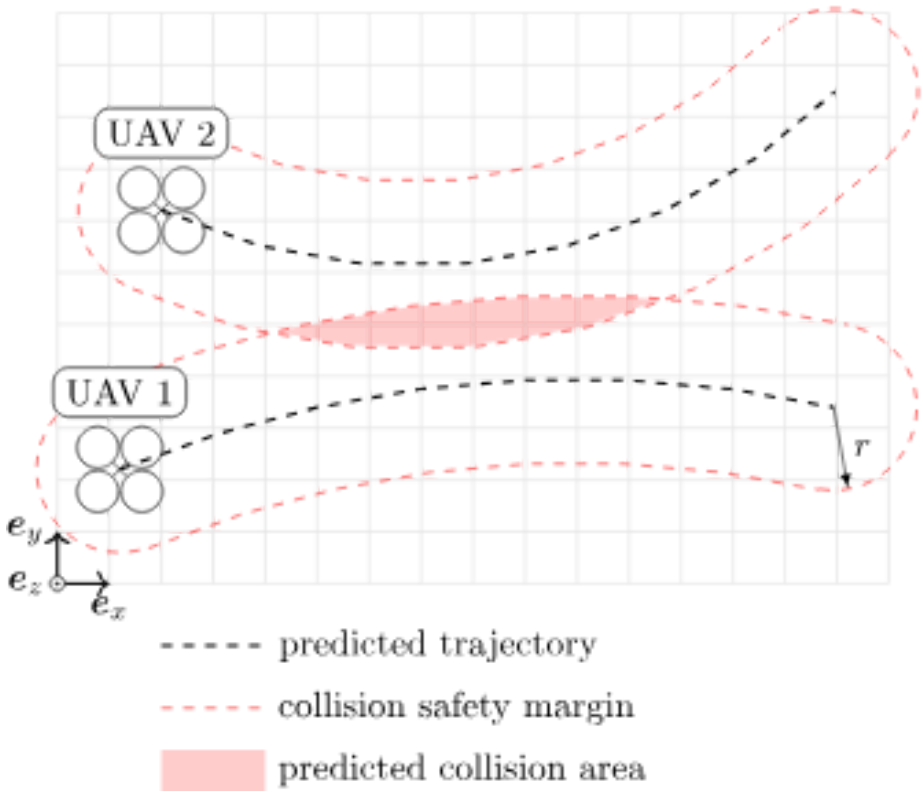
```
- name: "control_manager/goto_trajectory_start"
  type: std_srvs/Trigger
- name: "control_manager/start_trajectory_tracking"
  type: std_srvs/Trigger
- name: "control_manager/trajectory_reference"
  type: mrs_msgs/TrajectoryReferenceSrv
- name: "uav_manager/land_home"
  type: std_srvs/Trigger
- name: "control_manager/switch_controller"
  type: mrs_msgs/String
- name: "control_manager/switch_tracker"
  type: mrs_msgs/String
- name: "constraint_manager/set_constraints"
  type: mrs_msgs/String
- name: "control_manager/enable_callbacks"
  type: std_srvs/SetBool
```

2. **Location:**
   /opt/ros/noetic/share/mrs_uav_gazebo_simulation/models/mrs_robots_description/sdf/f450.sdf.jinja
   https://drive.google.com/file/d/1daL0oB-R3PjKOyh_Jw0-fFuuH2iBLsiR/view?usp=sharing
   Change the camera sensor angles as shown in the above link.

3. Location: /opt/ros/noetic/share/mrs_uav_gazebo_simulation/tmux/three_drones/session.yml
   Add camera processing as shown below in the link
   https://drive.google.com/file/d/1qSDDI2x5QRXWgBbRDEsZUp5h1qSo5qPm/view?usp=sharing

Command to use the camera
S rqt

Collision Avoidance



Collision Radius (r):
Each UAV inflates its predicted trajectory by a safety margin radius r (e.g., 5.0 m). A potential collision is detected if any two inflated trajectories overlap.

Altitude Change (correction):
When a collision is predicted, the lower-priority UAV increases its altitude by a defined correction value (e.g., 3.0 m) to avoid the other UAV.

UAV Priority Rules:
Priority is based on UAV names: the UAV with the lower number has higher priority (e.g., UAV1 has priority over UAV2). So if UAV1 is getting closer to UAV2, it is UAV2 that changes its altitude.

To make changes into the collision parameters go to location:
/opt/ros/noetic/share/mrs_uav_trackers/config/public/mpc_tracker.yaml

```
mrs_uav_trackers:

  mpc_tracker:

    wiggle:
      enabled: false
      amplitude: 0.5 # [m]
      frequency: 0.2 # [Hz]

# mrs collision avoidance
collision_avoidance:

enabled:true # disabling this will stop this UAV to react to others, but it will still transmit data to others
enabled_passively: true # disabling this will stop this uav to even transmit its data to others (only if enabled: false)
radius: 0.5 # [m]  # change this radius accordingly to your liking
correction: 2.0 # [m]   # altitude change
```

You can make changes accordingly to the above yaml code to change the threshold values of collision between the drones